

The Knowledge Level***

Allen Newell

*Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, PA 15213, U.S.A.*

Recommended by Daniel G. Bobrow

1. Introduction

This is the first presidential address of AAAI, the American Association for Artificial Intelligence. In the grand scheme of history, even the history of artificial intelligence (AI), this is surely a minor event. The field this scientific society represents has been thriving for quite some time. No doubt the society itself will make solid contributions to the health of our field. But it is too much to expect a presidential address to have a major impact.

So what is the role of the presidential address and what is the significance of the first one? I believe its role is to set a tone, to provide an emphasis. I think the role of the *first* address is to take a stand about what that tone and emphasis should be, to set expectations for future addresses and to communicate to my fellow presidents.

Only two foci are really possible for a presidential address: the state of the society or the state of the science. I believe the latter to be the correct focus. AAAI itself, its nature and its relationship to the larger society that surrounds it, are surely important.¹ However, our main business is to help AI become a science—albeit a science with a strong engineering flavor. Thus, though a president's address cannot be narrow or highly technical, it can certainly address a substantive issue. That is what I propose to do.

I wish to address the question of knowledge and representation. That is a little

*Presidential Address, American Association for Artificial Intelligence, AAAI80, Stanford University, 19 Aug 1980. Also published in the *AI Magazine* 2(2) 1981.

**This research was sponsored by the Defense Advanced Research Projects Agency (DOD), ARPA Order No.3597, monitored by the Air Force Avionics Laboratory Under Contract F33615-78-C-1551.

The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Defense Advanced Research Projects Agency or the US Government.

¹I have already provided some comments, as president, on such matters [25].

like a physicist wishing to address the question of radiation and matter. Such broad terms designate a whole arena of phenomena and a whole armada of questions. But comprehensive treatment is neither possible nor intended. Rather, such broad phrasing indicates an intent to deal with the subject in some basic way. Thus, the first task is to make clear the aspect of knowledge and representation of concern, namely, what is the nature of knowledge. The second task will be to outline an answer to this question. As the title indicates, I will propose the existence of something called the *knowledge level*. The third task will be to describe the knowledge level in as much detail as time and my own understanding permits. The final task will be to indicate some consequences of the existence of a knowledge level for various aspects of AI.

2. The Problem of Representation and Knowledge

2.1. The standard view

Two orthogonal and compatible basic views of the enterprise of AI serve our field, beyond all theoretical quibbles. The first is a geography of task areas. There is puzzle solving, theorem proving, game-playing induction, natural language, medical diagnosis, and on and on, with subdivisions of each major territory. AI, in this view, is an exploration, in breadth and in depth, of new territories of tasks with their new patterns of intellectual demands. The second view is the functional components that comprise an intelligent system. There is a perceptual system, a memory system, a processing system, a motor system, and so on. It is this second view that we need to consider to address the role of representation and knowledge.

Fig. 1. shows one version of the functional view, taken from [28], neither better nor worse than many others. An intelligent agent is embedded in a *task environment*; a *task statement* enters via a *perceptual* component and is encoded in an initial *representation*. Whence starts a cycle of activity in which a *recognition* occurs (as indicated by the *eyes*) of a method to use to attempt the problem. The method draws upon a memory of *general world knowledge*. In the course of such cycles, new methods and new representations may occur, as the agent attempts to solve the problem. The *goal structure*, a component we all believe to be important, does not receive its due in this figure, but no matter. Such a picture represents a convenient and stable decomposition of the *functions* to be performed by an intelligent agent, quite independent of particular implementations and anatomical arrangements. It also provides a convenient and stable decomposition of the entire scientific field into subfields. Scientists specialize in perception, or problem solving methods, or representation, etc.

It is clear to us all what *representation* is in this picture. It is the data structures that hold the problem and will be processed into a form that makes the solution available. Additionally, it is the data structures that hold the world

details, not on the main conceptions. Not everyone understands the necessary scientific role of such experiments in computational living, nor that standard experimental techniques cannot provide the same information. How else to explain, for example, the calls for controlled experimental validation that speech understanding will be useful to computer science? When that experiment of style is finally performed there will be no doubt at all. No standard experiment will be necessary. Indeed, none could have sufficed.

As an example related to the present paper, I have spent some effort recently in describing what Herb Simon and I have called the “*Physical symbol system hypothesis*” [26, 29]. This hypothesis identifies a class of systems as embodying the essential nature of symbols and as being the necessary and sufficient condition for a generally intelligent agent. Symbol systems turn out to be universal computational systems, viewed from a different angle. For my point here, the important feature of this hypothesis is that it grew out of the practice in AI—out of the development of list processing languages and Lisp, and out of the structure adopted in one AI program after another. We in AI were led to an adequate notion of a symbol by our practice. In the standard catechism of science, this is not how great ideas develop. Major ideas occur because great scientists discover (or invent) them, introducing them to the scientific community for testing and elaboration. But here, working scientists have evolved a new major scientific concept, under partial and alternative guises. Only gradually has it acquired its proper name.

The notions of knowledge and representation about to be presented also grow out of our practice. At least, so I assert. That does not give them immunity from criticism, for in listening for these lessons I may have a tin ear. But in so far as they are wanting, the solution lies in more practice and more attention to what emerges there as pragmatically successful. Of course, the message will be distorted by many things, e.g., peculiar twists in the evolution of computer science hardware and software, our own limitations of view, etc. But our practice remains a source of knowledge that cannot be obtained from anywhere else. Indeed, AI as a field is committed to it. If it is fundamentally flawed, that will just be too bad for us. Then, other paths will have to be found from elsewhere to discover the nature of intelligence.

3. The Knowledge Level

I am about to propose the existence of something called the *knowledge level*, within which knowledge is to be defined. To state this clearly, requires first reviewing the notion of computer systems levels.

3.1. Computer system levels

Fig. 2 shows the standard hierarchy, familiar to everyone in computer science. Conventionally, it starts at the bottom with the *device level*, then up to the

circuit level, then the *logic level*, with its two sublevels, *combinatorial and sequential circuits*, and the *register-transfer level*, then the *program level* (referred to also as the *symbolic level*) and finally, at the top, the *configuration level* (also called the *PMS* or *Processor-Memory-Switch level*). We have drawn the configuration level to one side, since it lies directly above both the symbol level and the register-transfer level.

The notion of levels occurs repeatedly throughout science and philosophy, with varying degrees of utility and precision. In computer science, the notion is quite precise and highly operational. Table 1 summarizes its essential attributes. A level consists of a *medium* that is to be processed, *components* that provide primitive processing, *laws of composition* that permit components to be assembled into *systems*, and *laws of behavior* that determine how system behavior depends on the component behavior and the structure of the system. There are many variant instantiations of a given level, e.g., many programming systems and machine languages and many register-transfer systems.⁴

Each level is defined in two ways. First, it can be defined autonomously, without reference to any other level. To an amazing degree, programmers need not know logic circuits, logic designers need not know electrical circuits, managers can operate at the configuration level with no knowledge of programming, and so forth. Second, each level can be reduced to the level below. Each aspect of a level—medium, components, laws of composition and behavior—can be defined in terms of systems at the level next below. The *architecture* is the name we give to the register-transfer level system that defines a symbol (programming) level, creating a machine language and making it run as described in the programmers manual for the machine. Neither of these two definitions of a level is the more fundamental. It is essential that they both exist and agree.

Some intricate relations exist between and within levels. Any instantiation of a level can be used to create any instantiation of the next higher level. Within each level, systems hierarchies are possible, as in the subroutine hierarchy at the programming level. Normally, these do not add anything special in terms of the computer system hierarchy itself. However, as we all know, at the program level it is possible to construct any instantiation within any other instantiation (modulo some rigidity in encoding one data structure into another), as in creating new programming languages.

There is no need to spin out the details of each level. We live with them every day and they are the stuff of architecture textbooks [1], machine manuals and digital component catalogues, not research papers. However, it is noteworthy how radically the levels differ. The medium changes from electrons and magnetic domains at the device level, to current and voltage at the circuit level,

⁴Though currently dominated by electrical circuits, variant circuit level instantiations also exist, e.g., fluidic circuits.

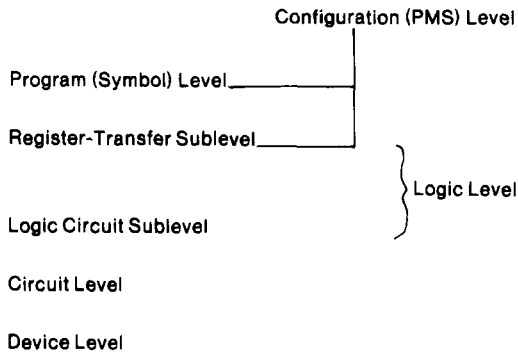


FIG. 2. Computer system levels.

to bits at the logic level (either single bits at the logic circuit level or bit vectors at the register-transfer level), to symbolic expressions at the symbol level, to amounts of data (measured in data bits) at the configuration level. System characteristics change from continuous to discrete processing, from parallel to serial operation, and so on.

Despite this variety, all levels share some common features. Four of these, though transparently obvious, are important to us.

Point 1. Specification of a system at a level always determines completely a definite behavior for the system at that level (given initial and boundary conditions).

Point 2. The behavior of the total system results from the local effects of each component of the system processing the medium at its inputs to produce its outputs.

Point 3. The immense variety of behavior is obtained by system structure, i.e., by the variety of ways of assembling a small number of component types (though perhaps a large number of instances of each type).

TABLE 1. Defining aspects of a computer system level

Aspects	Register-transfer level	Symbol level
Systems	Digital systems	Computers
Medium	Bit vectors	Symbols, expressions
Components	Registers	Memories
	Functional units	Operations
Composition laws	Transfer path	Designation, association
Behavior laws	Logical operations	Sequential interpretation

Point 4. The medium is realized by state-like properties of matter, which remain passive until changed by the components.

Computer systems levels are not simply levels of abstraction. That a system has a description at a given level does not necessarily imply it has a description at higher levels. There is no way to abstract from an arbitrary electronic circuit to obtain a logic-level system. There is no way to abstract from an arbitrary register-transfer system to obtain a symbol-level system. This contrasts with many types of abstraction which can be uniformly applied, and thus have a certain optional character (as in abstracting away from the visual appearance of objects to their masses). Each computer system level is a *specialization* of the class of systems capable of being described at the next lower level. Thus, it is a priori open whether a given level has any physical realizations.

In fact, computer systems at all levels are realizable, reflecting indirectly the structure of the physical world. But more holds than this. Computer systems levels are realized by *technologies*. The notion of a technology has not received the conceptual attention it deserves. But roughly, given a specification of a particular system at a level, it is possible to construct by *routine* means a physical system that realizes that specification. Thus, systems can be obtained to specification within limits of time and cost. It is not possible to invent arbitrarily additional computer system levels that nestle between existing levels. Potential levels do not become technologies, just by being thought up. Nature has a say in whether a technology can exist.

Computer system levels are *approximations*. All of the above notions are realized in the real world only to various degrees. Errors at lower levels propagate to higher ones, producing behavior that is not explicable within the higher level itself. Technologies are imperfect, with constraints that limit the size and complexity of systems that can actually be fabricated. These constraints are often captured in design rules (e.g., fan-out limits, stack-depth limits, etc), which transform system design from routine to problem solving. If the complexities become too great, the means of system creation no longer constitute a technology, but an arena of creative invention.

We live quite comfortably with imperfect system levels, especially at the extremes of the hierarchy. At the bottom, the device level is not complete, being used only to devise components at the circuit level. Likewise, at the top, the configuration level is incomplete, not providing a full set of behavioral laws. In fact, it is more nearly a pure level of abstraction than a true system level. This accounts for both symbol level and register transfer level systems having configuration (PMS) level abstractions (see [2] for a PMS approach to the register-transfer level).

These levels provide ways of describing computer systems; they do not provide ways of describing their environments. This may seem somewhat unsatisfactory, because a level does not then provide a general closed description of an entire universe, which is what we generally expect (and get) from a

level of scientific description in physics or chemistry. However, the situation is understandable enough. System design and analysis requires only that the interface between the environment and the system (i.e., the inner side of the transducers) be adequately described in terms of each level, e.g., as electrical signals, bits, symbols or whatever. Almost never does the universe of system plus environment have to be modeled in toto, with the structure and dynamics of the environment described in the same terms as the system itself. Indeed, in general no such description of the environment in the terms of a given computer level exists. For instance, no register-transfer level description exists of the airplane in which an airborne computer resides. Computer system levels describe the internal structure of a particular class of systems, not the structure of a total world.

To sum up, computer system levels are a reflection of the nature of the physical world. They are not just a point of view that exists solely in the eye of the beholder. This reality comes from computer system levels being genuine specializations, rather than being just abstractions that can be applied uniformly.

3.2. A new level

I now propose that there does exist yet another system level, which I will call the *knowledge level*. It is a true systems level, in the sense we have just reviewed. The thrust of this paper is that distinguishing this level leads to a simple and satisfactory view of knowledge and representation. It dissolves some of the difficulties and confusions we have about this aspect of artificial intelligence.

A quick overview of the knowledge level, with an indication of some of its immediate consequences, is useful before entering into details.

The system at the knowledge level is the *agent*. The components at the knowledge level are *goals*, *actions*, and *bodies*. Thus, an agent is composed of a set of actions, a set of goals and a body. The medium at the knowledge level is *knowledge* (as might be suspected). Thus, the agent processes its knowledge to determine the actions to take. Finally, the behavior law is the *principle of rationality*: Actions are selected to attain the agent's goals.

To treat a system at the knowledge level is to treat it as having some knowledge and some goals, and believing it will do whatever is within its power to attain its goals, in so far as its knowledge indicates. For example:

- “She knows where this restaurant is and said she'd meet me here. I don't know why she hasn't arrived.”

- “Sure, he'll fix it. He knows about cars.”

- “If you know that $2 + 2 = 4$, why did you write 5?”

The knowledge level sits in the hierarchy of systems levels immediately above the symbol level, as Fig. 3 shows. Its components (actions, goals, body)

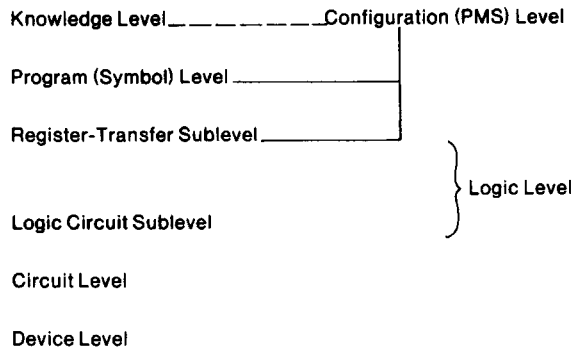


FIG. 3. New version of computer system levels.

and its medium (knowledge) can be defined in terms of systems at the symbol level, just as any level can be defined by systems at the level one below. The knowledge level has been placed side by side with the configuration level. The gross anatomical description of a knowledge-level system is simply (and only) that the agent has as parts bodies of knowledge, goals and actions. They are all connected together in that they enter into the determination of what actions to take. This structure carries essentially no information; the specification at the knowledge level is provided entirely by the content of the knowledge and the goals, not by any structural way they are connected together. In effect, the knowledge level can be taken to have a degenerate configuration level.

As is true of any level, although the knowledge level can be constructed from the level below (i.e., the symbol level), it also has an autonomous formulation as an independent level. Thus, knowledge can be defined independent of the symbol level, but can also be reduced to symbol systems.

As the Fig. 3 makes evident, the knowledge level is a separate level from the symbol level. Relative to the existing view of the computer systems hierarchy, the symbol level has been split in two, one aspect remaining as the symbol level proper and another aspect becoming the knowledge level. The description of a system as exhibiting intelligent behavior still requires both levels, as we shall see. Intelligent systems are not to be described exclusively in terms of the knowledge level.

To repeat the final remark of the prior section: Computer system levels really exist, as much as anything exists. They are not just a point of view. Thus, to claim that the knowledge level exists is to make a scientific claim, which can range from dead wrong to slightly askew, in the manner of all scientific claims. Thus, the matter needs to be put as a hypothesis:

The Knowledge Level Hypothesis. *There exists a distinct computer systems level, lying immediately above the symbol level, which is characterized by knowledge as the medium and the principle of rationality as the law of behavior.*

Some preliminary feeling for the nature of knowledge according to this hypothesis can be gained from the following remarks.

- Knowledge is intimately linked with rationality. Systems of which rationality can be posited can be said to have knowledge. It is unclear in what sense other systems can be said to have knowledge.

- Knowledge is a competence-like notion, being a potential for generating action.⁵

- The knowledge level is an approximation. Nothing guarantees how much of a system's behavior can be viewed as occurring at the knowledge level. Although extremely useful, the approximation is quite imperfect, not just in degree but in scope.

- Representations exist at the symbol level, being systems (data structures and processes) that realize a body of knowledge at the knowledge level.

- Knowledge serves as the specification of what a symbol structure should be able to do.

- Logics are simply one class of representations among many, though uniquely fitted to the analysis of knowledge and representation.

4. The Details of the Knowledge Level

We begin by defining the knowledge level autonomously, i.e., independently of lower system levels. Table 1 lists what is required, though we will take up the various aspects in a different order: first, the structure, consisting of the system, components and laws for composing systems; second, the laws of behavior (the law of rationality); and third, the medium (knowledge). After this we will describe how the knowledge level reduces to the symbol level.

Against the background of the common features of the familiar computer system levels listed earlier (Section 3.1), there will be four surprises in how the knowledge level is defined. These will stretch the notion of system level somewhat, but will not break it.

4.1. The structure of the knowledge level

An agent (the system at the knowledge level), has an extremely simple structure, so simple there is no need even to picture it.

First, the agent has some *physical body* with which it can act in the environment (and be acted upon). We talk about the body as if it consists of a set of *actions*, but that is only for simplicity. It can be an arbitrary physical system with arbitrary modes of interaction with its environment. Though this

⁵How almost interchangeable the two notions might be can be seen from a quotation from Chomsky [9, p. 315]: "In the past I have tried to avoid, or perhaps evade the problem of explicating the notion 'knowledge of language' by using an invented technical term, namely the term 'competence' in place of 'knowledge'".